JackThomasNortheastern / cs5004-assignments

Code Issues Pull requests Actions Projects Security Insights

main cs5004-assignments / instructions / README_A5.md

JackThomasNortheastern Rename README_A5 to README_A5.md 62a06fc · now

166 lines (120 loc) · 9.94 KB

Preview Code Blame Raw

GUI Addition - Instructions

For this assignment, you are going to pick one of your previous assignments and add a GUI to it. You can pick Homework 03, 05, or 07, and you will revamp the assignment to follow the MVC design pattern.

"Living with your code" is a common expression used in computer science. It means when you write code you should expect to have to come back to it and work with it again. This assignment is meant to give you a sense of that by having you use the code from your previous assignment, while adding more features to that code base.

Learning Objectives

- Understand the process of adding a GUI to an existing program
- Modify existing code to work with multiple types of views / adding new features.

Instructions

You will need to take a previous assignment (03, 05, or 07) and convert it into the MVC design pattern. Additionally, you will need to add a GUI to the program, which means this assignment is a culmination of everything you learned up until this point.

Suggested Parts:

1. Copy your code from the previous assignment into this repository.
2. Modify your application making sure you take steps to test each modification *AS YOU MODIFY*.
3. Work on designing your GUI. Grading for the GUI will be based on screenshots and the TAs running your program if they don't see obvious tests/clarity in what you are providing.

> Tip
>
> We have once again provided a sample working program, that includes both the console and GUI version. This is a very basic GUI for DNInfo, you do not (nor should you) copy the design, but instead use it as a reference for minimum functionality. The weird X and . characters are just because because it is a demo version.
>
> From the sample_working directory.
>
> To run the GUI version, you can use the following command
>
> ```
> bin/DNInfoGUI -g
> ```
>
> or if on windows
>
> ```
> bin\DNInfoGUI.bat -g
> ```
>
> To run the console version, you can use the following command
>
> ```
> bin/DNInfoGUI -c
> ```
>
> or if on windows
>
> ```
> bin\DNInfoGUI.bat -c
> ```
>
> as with before you can run help by doing
>
> ```
> bin/DNInfoGUI -h
> ```
>
> or if on windows

```
bin\DNInfoGUI.bat -h
```

> It is worth noting the old DNINfo app functionality (command line arguments) are all still there. We only added the views in this assignment.

🔥 Task 1: Design

Before you start writing, it is important to think about design. You DO NOT have to be perfect in your design, so we will come back to this step a few times.

1. Go to DesignDocument.md and fill out (ONLY) the initial design sections. Make sure to start with the end design of whatever assignment you are modifying!

> Tip
>
> You are free to use mermaid or any other UML tools you want, just make sure if you are using another UML tool, you properly link the image in the markdown file.

🔥 Task 2: Implement by Test Driven Development

After your initial design, you should seek to follow the TDD process. This means you should write tests first, and then implement the code to pass those tests. Or better stated, you should write *ONE* test first, implement, and repeat until you have written all your tests.

1. Figure out a number of tests by brainstorming (done in design)
2. Write **one** test
3. Write **just enough** code to make that test pass
4. Refactor/update as you go along
5. Repeat steps 2-4 until you have all the tests passing/fully built program

Note: you often don't know all the tests as you write. As such, it is alright to continue to expand your list. This is where people get stuck on TDD. They think they have to know **all** the tests before they start. You don't. You just need to know the next test, and then at the end you double check you have covered all code paths and have full coverage.

> Caution

> Make sure to commit as you development. The bare minimum commits would be after every test, but you probably will have additional commits especially at the beginning.

Testing GUIs

When it comes to testing GUIs you often have to document how you tested, and what you did. Use the GuiTestingHistory.md document to document including screen shots of your testing.

💡 Implementation Tips

Here are a few tips to help you out.

- The GUI is *always* easier to write in parts and test those parts. For example, we created the jframe, ran the program, and slowly started adding components. After we had the look at feel we wanted, we then added the functionality (the listeners).
  - To design a GUI, you can use a GUI designer, or this is actually a good place for Copilot or other AI Tool. For example, you could have the following prompt. `Create class that extends JPanel. Inside that panel, I want a label that says 'hostname', a text field that is 20 characters long, a button that says 'search'. The search button should be the same length of the text field, and sits under it. The label should be above the text field. The panel should be 200x200 with the components centered. Each component should have a margin of 10 pixels.`
  - This helps find the java swing awt components but **often has errors**. We then would spend time modifying the provided code, but at least, it gives you a starting point. DO NOT take it as is! Make changes, understand what it is creating.
- For a file chooser/dialog, you can use the JFileChooser. This is a common component in Java Swing. Here is an example of our FileChooser.

```
JFileChooser fileChooser = new JFileChooser();
fileChooser.setAcceptAllFileFilterUsed(false);
fileChooser.setCurrentDirectory(new File(System.getProperty("user.dir"))
fileChooser.setDialogTitle("Export List");
fileChooser.setFileFilter(new FileNameExtensionFilter("XML (*.xml)", "xm
fileChooser.addChoosableFileFilter(new FileNameExtensionFilter("JSON (*.
fileChooser.addChoosableFileFilter(new FileNameExtensionFilter("CSV (*.c
fileChooser.addChoosableFileFilter(new FileNameExtensionFilter("Text (*.

int userSelection = fileChooser.showSaveDialog(this);

if (userSelection == JFileChooser.APPROVE_OPTION) {
// do something
}
```

IMPORTANT!

HAVE FUN! You can play with features. Maybe you want to use the latitude/ longitude to display a location on a map. You are not limited to the bare minimum for this GUI. While we will only grade making sure you have all the minimum features, you can add more. If you go above and beyond, make sure to document it in the final design document - so that the TAs (and thus instructor) can see it! We also encourage sharing some of your final screen shots in Teams if you add a neat feature you want to share.

🔥 Task 3: Finish Design Document

By this point, your design has probably changed (very few people have perfect designs the first iteration). Update your design document with the final design in the "final design" section. We want to see the history of your first design to your final design. That is a good thing.

🔥 Task 4: Build a Manual

Inside of manual You will need to build a 'manual' for your application. It presents screenshots of the GUI, and explains how to use the program in a logical manner. Reading this manual one should have a sense of the full features of the program.

Submission

When you are completed, you need to submit your code to gradescope. Go back to Canvas, and click through the link that takes you to the Gradescope assignment. When you submit, you will actually need to pull from your github repository in the dialog that appears. It only pulls the most recent submission, and if you make an update to the repository after you submit, you will need to resubmit to get the latest version in gradescope.

📝 Grading Rubric

1. Learning (MG)
    - Code is fully documented using javadoc commenting style
    - Code has tests well organized for model layer, controller
2. Approaching (MG)
    - Follows style guidelines (suggestion run style check locally)
    - GuiTestingHistory.md - View has evidence of testing (via screenshots, and documentation on how you tested)
3. Meets (MG)
    - README.md is filled out (name, github repo, etc)
        - With out the link to your repo, the TAs won't grade the rest!
    - DesignDocument (INITIAL) sections are filled out
    - Manual is so someone can easily use the program, showing screenshots of features.
4. Exceeds (MG)
    - Code is DRY (Don't Repeat Yourself)
        - Including making use of helping/utility classes to reduce duplication.
    - Student uses proper inheritance without duplication
    - Methods include tests for edge cases in addition to happy path
    - Design document (FINAL) sections are filled out
        - The notation needs to be correct, and the TAs will double check the final design matches the final implementation.

Legend:

- AG - Auto-graded
- MG - Manually graded

Submission Reminder 🚨

For manually graded elements, we only guarantee time to submit for a regrade IF you submit by the DUE DATE. Submitting late may mean it isn't possible for the MG to be graded before the AVAILABLE UNTIL DATE, removing any windows for you to resubmit in time. While it will be graded, it is always best to submit by the due date, so you have full opportunity to improve your grade.

If you need a reminder about the grading policy, please review the syllabus and the canvas page on 'formative/summative' grading. This class uses a unique grading system that will allow you to be flexible with due dates and multiple resubmissions (if you submit with time for TAs to give feedback), but we also ask that you continue to work on the assignment until you get a full grade.

> Caution
>
> As this is the last assignment, your submission window between due date and available until date may be shorter! Please take that into account.

No Autograder - Still submit to gradescope

This assignment has no autograder. As such, you will need to make sure you are following the rubric and the instructions. If you have questions, please ask in the Teams channel. The TA will grade them as they come in, giving feedback as they go.